

NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Affiliated to Pokhara University

A Data Analysis Case Study On Student Placement Prediction

Course Name: Data Science And Analytics

Course Code: CMP 360

Program: B.E. Information Technology

Semester: Sixth Semester

Submitted By:

Anuskha josi [231542]

DipKiran Limbu [231512]

Sumit kumar Das [231536]

Suraj Kumar Jha [231537]

Submitted To: Er. Manil Baidhya

Date: 11 July 2026



Department of Information Technology

**NEPAL COLLEGE OF
INFORMATION TECHNOLOGY**

Balkumari, Lalitpur, Nepal

Contents

1	Problem Statement	2
2	Information of the Data	2
3	Source of Data	3
4	Data Preprocessing	3
4.1	a. Data Cleaning	3
4.2	b. Data Standardization	7
4.3	c. Data Transformation: Binning	8
4.4	d. Handling Categorical Data: Encoding Techniques	9
4.5	e. Outlier Detection	10
5	Data Visualization	12
5.1	Univariate Distributions	13
5.2	Placement vs. Categorical Features	15
5.3	Placement vs. Numeric Features	18
5.4	Correlation Heatmap	20
6	Statistical Analysis	20
6.1	a. Hypothesis Testing	20
6.2	b. Correlation Analysis	23
7	Results and Discussion	24
8	Challenges and Future Enhancement	25

1 Problem Statement

Campus placement outcomes are of central concern to students, academic institutions, and recruiters alike. Institutions want to understand which student attributes — demographic (age, gender, city), academic (CGPA, degree/profession level), or otherwise — are meaningfully associated with whether a student is placed in a job after graduation. Understanding these relationships helps institutions design better placement support programs, and helps students understand what factors genuinely affect employability outcomes versus common assumptions and myths.

This case study analyzes a dataset of 1,100 student records containing demographic and academic attributes alongside a binary placement outcome. The objective is to clean and prepare the raw data, explore it visually, and apply formal statistical tests to answer the central question:

“Which student attributes, if any, are statistically associated with placement status?”

The analysis follows a complete data-science workflow — data cleaning, transformation, encoding, outlier handling, visualization, and hypothesis testing — culminating in evidence-based conclusions about the drivers (or absence thereof) of student placement in this dataset.

2 Information of the Data

The raw dataset contains 1,100 rows and 7 columns, mixing categorical (nominal) and numerical (continuous) features, along with a binary target variable. Table 1 summarizes each column.

Table 1: Dataset column summary

Column	Type	Description	Missing (raw)
name	Categorical (text)	Student’s first name	85 (7.7%)
city	Categorical (nominal)	City of residence (asgard, wakanda, gotham, purgatory)	184 (16.7%)
gender	Categorical (binary)	Gender of the student (male / female)	99 (9.0%)
profession	Categorical (ordinal)	Highest degree pursued (bachelor / masters / phd)	169 (15.4%)
age	Numeric (continuous)	Student’s age in years	112 (10.2%)
cgpa	Numeric (continuous)	Cumulative Grade Point Average (0–10 scale)	132 (12.0%)
placed	Binary (target)	Placement outcome (1 = placed, 0 = not placed)	0 (0%)

The dataset shape was verified using `df.shape` and `df.info()`, confirming 1,100 entries across 7 columns with the null counts and dtypes shown below.

```

> df = pd.read_csv("my_data (1).csv")
> print(df.shape)
> df.head()

[2]

... (1100, 7)

...
   name  city gender profession  age  cgpa  placed
0  moriarity  NaN  female      phd   28.0   5.94      1
1  moriarity  asgard   NaN  bachelor   50.0   8.55      0
2   holmes   NaN  female  masters   18.0   5.56      0
3     sam   NaN   male  bachelor   25.0   8.57      1
4     sam   NaN   male  bachelor   19.0   8.76      1

> df.info()

[3]

...
<class 'pandas.DataFrame'>
RangeIndex: 1100 entries, 0 to 1099
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   name        1009 non-null    str
1   city        905 non-null     str
2   gender      994 non-null     str
3   profession  921 non-null     str
4   age         982 non-null     float64
5   cgpa        962 non-null     float64
6   placed      1100 non-null    int64
dtypes: float64(2), int64(1), str(4)
memory usage: 60.3 KB

```

Figure 1: Dataset shape, sample rows, and `df.info()` output

The target variable `placed` is well balanced, with 521 students not placed (50.3%) and 514 students placed (49.7%) after cleaning — an almost even 50/50 split, which is favorable for unbiased statistical comparison between the two groups.

3 Source of Data

The dataset (`my_data.csv`) was provided as a course/assignment dataset for the purpose of this case study. It is a structured, tabular dataset resembling a typical student-placement record used in academic exercises on data cleaning and statistical analysis.

It is worth noting that the categorical values in the `name` column (e.g., moriarity, holmes, sherlock, sam, dean, castiel, bobby) and in the `city` column (asgard, wakanda, gotham, purgatory) are drawn from popular fictional universes rather than real-world identifiers. This strongly suggests the dataset is synthetically generated / anonymized for educational purposes rather than sourced from a real institutional placement cell. This context is relevant when interpreting results — patterns found here characterize the dataset itself and the methodology applied to it, not any real population of students.

4 Data Preprocessing

4.1 a. Data Cleaning

Duplicate Records

Duplicate rows were checked with `df.duplicated().sum()`. The raw dataset ($1,100 \times 7$) contained 65 fully duplicate rows (5.9% of the data), which were removed using `df.drop_duplicates()`. A later verification pass, run after feature engineering (standardization, binning, and encoding had expanded the frame to 16 columns), confirmed 1,032 unique rows with zero remaining duplicates.

(1) Handling Duplicates and Dropping Unnecessary Columns

```
print(f"Total no of rows and columns before Dropping the Duplicates data : {df.shape}")

print("\nTotal Duplicate rows:", df.duplicated().sum())

df[df.duplicated()]
```

[4] ... Total no of rows and columns before Dropping the Duplicates data : (1100, 7)

Total Duplicate rows: 65

	name	city	gender	profession	age	cgpa	placed
1000	castiel	asgard	male	masters	24.0	7.71	1
1001	NaN	wakanda	male	bachelor	27.0	7.90	0
1005	moriarity	asgard	male	phd	25.0	NaN	1
1006	sherlock	gotham	male	masters	26.0	7.55	1
1008	dean	wakanda	male	bachelor	20.0	9.31	0
...	...	...	...	...	...	...	...
1094	moriarity	gotham	male	NaN	26.0	7.96	0
1095	holmes	NaN	male	masters	26.0	8.92	1
1096	moriarity	wakanda	male	masters	19.0	9.01	0
1097	sam	asgard	male	bachelor	30.0	7.88	0
1099	sam	asgard	NaN	masters	24.0	8.94	1

65 rows × 7 columns

Figure 2: Detecting the 65 duplicate rows before removal

```
df = df.drop_duplicates()

print(f"Total no of rows and columns After Dropping the Duplicates data : {df.shape}")

# checking for duplicate data:
print(df.duplicated().sum())
```

[38] ✓ 0.0s

... Total no of rows and columns After Dropping the Duplicates data : (1032, 16)

0

Figure 3: Shape confirmation after dropping duplicates

Missing Value Detection and Treatment

All seven columns except the target were found to contain missing values, ranging from 8.2% (name) to 17.8% (city). Given the scale of missingness, rows were not dropped outright (this would have discarded a large fraction of the dataset); instead, column-appropriate imputation was applied:

- **Numerical columns** (age, cgpa): imputed with the column **median**, which is robust to skew and outliers.
- **Categorical columns** (city, gender, profession): imputed with the column **mode** (most frequent category).

- **name:** imputed with the placeholder label “Unknown”, since it carries no analytical/statistical value beyond identification.

▼ (2) Detect and Handle Missing Values

```
# checking the columns for missing values
df.isnull().sum()
```

```
[7] ✓ 0.0s
```

```
... name      85
    city     184
    gender    99
    profession 169
    age       112
    cgpa      132
    placed     0
    dtype: int64
```

We should not simply delete all these rows because that would remove a significant amount of data.

```
missing_percentage = (df.isnull().sum() / len(df)) * 100
print(missing_percentage)
```

```
[8] ✓ 0.0s
```

```
... name      8.212560
    city     17.777778
    gender    9.565217
    profession 16.328502
    age       10.821256
    cgpa      12.753623
    placed    0.000000
    dtype: float64
```

Figure 4: Missing value counts and percentage-missing per column

handling the numerical coulmns like age and cgpa

```
df.dtypes
```

```
[9] ✓ 0.0s
```

```
... name      str
    city      str
    gender     str
    profession str
    age      float64
    cgpa     float64
    placed    int64
    dtype: object
```

```
df["age"] = df["age"].fillna(df["age"].median())
df["cgpa"] = df["cgpa"].fillna(df["cgpa"].median())
```

```
[10] ✓ 0.0s
```

Figure 5: Median imputation for age and cgpa

```

Handling the categorical columns: name, city, gender, profession

df["city"] = df["city"].fillna(df["city"].mode()[0])
df["gender"] = df["gender"].fillna(df["gender"].mode()[0])
df["profession"] = df["profession"].fillna(df["profession"].mode()[0])
[11] ✓ 0.0s

df["name"] = df["name"].fillna("Unknown")
[12] ✓ 0.0s

# Checking for missing values after filling

df.isnull().sum()
[13] ✓ 0.0s

...
name      0
city      0
gender    0
profession 0
age       0
cgpa      0
placed    0
dtype: int64

```

Figure 6: Mode imputation for `city/gender/profession` and placeholder fill for `name`

After imputation, `df.isnull().sum()` confirmed zero missing values across all columns.

Data Type Correction

`df.dtypes` revealed that `age` was stored as `float64` despite representing whole years. It was cast to an integer type using `df["age"] = df["age"].astype(int)` for consistency and readability.

```

(3) Cheking for the correct datatype in each column

df.dtypes
[14] ✓ 0.0s

...
name      str
city      str
gender    str
profession str
age       float64
cgpa      float64
placed    int64
dtype: object

df["age"] = df["age"].astype(int)
df.dtypes
[15] ✓ 0.0s

...
name      str
city      str
gender    str
profession str
age       int64
cgpa      float64
placed    int64
dtype: object

```

Figure 7: Correcting the dtype of the `age` column

4.2 b. Data Standardization

The numerical features `age` and `cgpa` were standardized using scikit-learn's `StandardScaler`, which rescales values to zero mean and unit variance (z -scores). This step is essential before running distance- or scale-sensitive analyses, and was verified by re-computing the mean (≈ 0) and standard deviation ($= 1.0$) of the resulting `age_standardized` and `cgpa_standardized` columns.

```

Standardization

> ✓
scaler = StandardScaler()

# Standardize Age
df["age_standardized"] = scaler.fit_transform(df[["age"]])

# Standardize CGPA
df["cgpa_standardized"] = scaler.fit_transform(df[["cgpa"]])

print("First 5 rows after standardization:")
print(df.head())

print("\nMean after Standardization:")
print(df[["age_standardized", "cgpa_standardized"]].mean())

print("\nStandard Deviation after Standardization:")
print(df[["age_standardized", "cgpa_standardized"]].std(ddof=0))

```

First 5 rows after standardization:

	name	city	gender	profession	age	cgpa	placed	age_standardized	\
0	moriarity	wakanda	female	phd	28	5.94	1	0.034603	
1	moriarity	asgard	male	bachelor	42	8.55	0	1.884797	
2	holmes	wakanda	female	masters	18	5.56	0	-1.286964	
3	sam	wakanda	male	bachelor	25	8.57	1	-0.361867	
4	sam	wakanda	male	bachelor	19	8.76	1	-1.154807	

cgpa_standardized

0	-1.168702
1	0.661708
2	-1.435199
3	0.675734
4	0.808983

Mean after Standardization:

age_standardized	2.540104e-16
cgpa_standardized	-1.235726e-16

dtype: Float64

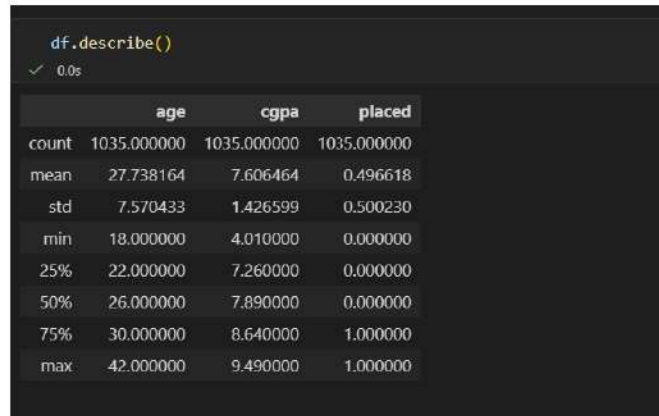
Standard Deviation after Standardization:

age_standardized	1.0
cgpa_standardized	1.0

dtype: float64

Figure 8: `StandardScaler` applied to `age` and `cgpa`, with verification statistics

For reference, the descriptive statistics of the cleaned (pre-standardization) numeric columns were:



```
df.describe()
```

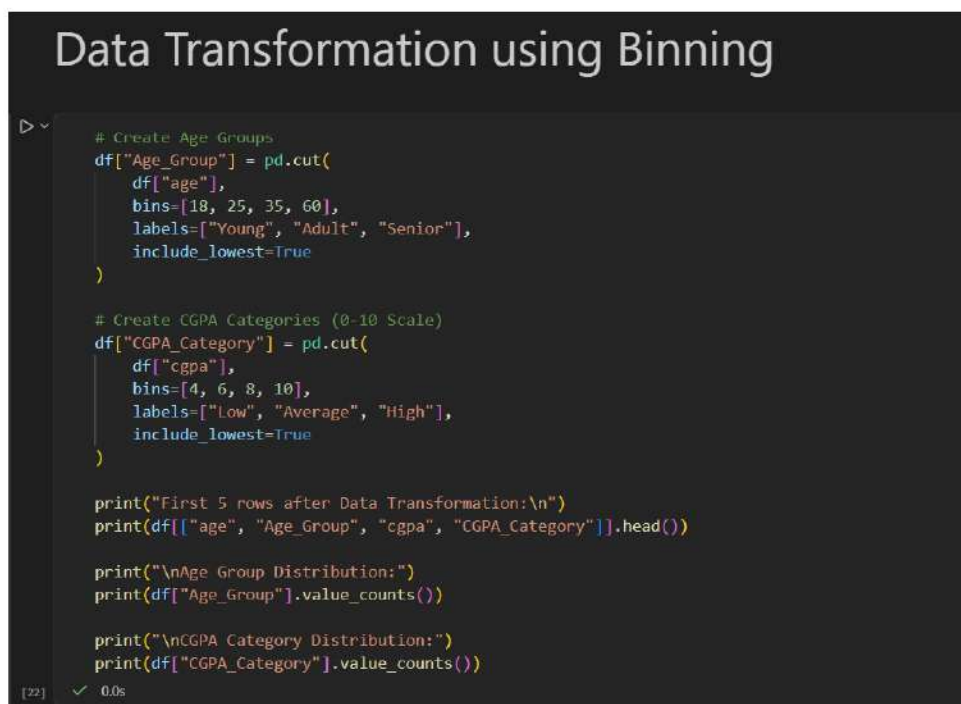
	age	cgpa	placed
count	1035.000000	1035.000000	1035.000000
mean	27.738164	7.606464	0.496618
std	7.570433	1.426599	0.500230
min	18.000000	4.010000	0.000000
25%	22.000000	7.260000	0.000000
50%	26.000000	7.890000	0.000000
75%	30.000000	8.640000	1.000000
max	42.000000	9.490000	1.000000

Figure 9: `df.describe()` after cleaning, before standardization

4.3 c. Data Transformation: Binning

To support categorical-style analysis (e.g., count plots, chi-square tests) alongside the continuous variables, two new binned features were engineered using `pd.cut()`:

- **Age_Group:** Young (18–25), Adult (25–35), Senior (35–60).
- **CGPA_Category:** Low (4–6), Average (6–8), High (8–10).



```

Data Transformation using Binning

# Create Age Groups
df["Age_Group"] = pd.cut(
    df["age"],
    bins=[18, 25, 35, 60],
    labels=["Young", "Adult", "Senior"],
    include_lowest=True
)

# Create CGPA Categories (0-10 Scale)
df["CGPA_Category"] = pd.cut(
    df["cgpa"],
    bins=[4, 6, 8, 10],
    labels=["Low", "Average", "High"],
    include_lowest=True
)

print("First 5 rows after Data Transformation:\n")
print(df[["age", "Age_Group", "cgpa", "CGPA_Category"]].head())

print("\nAge Group Distribution:")
print(df["Age_Group"].value_counts())

print("\nCGPA Category Distribution:")
print(df["CGPA_Category"].value_counts())

```

Figure 10: Binning code for `Age_Group` and `CGPA_Category`

First 5 rows after Data Transformation:

	age	Age_Group	cgpa	CGPA_Category
0	28	Adult	5.94	Low
1	42	Senior	8.55	High
2	18	Young	5.56	Low
3	25	Young	8.57	High
4	19	Young	8.76	High

Age Group Distribution:

Age_Group

Young 437

Adult 398

Senior 200

Name: count, dtype: int64

CGPA Category Distribution:

CGPA_Category

High 423

Average 412

Low 200

Name: count, dtype: int64

Figure 11: Resulting distribution of Age_Group and CGPA_Category

The resulting distribution is fairly balanced: 437 Young, 398 Adult, and 200 Senior students; and 423 High, 412 Average, and 200 Low CGPA students.

4.4 d. Handling Categorical Data: Encoding Techniques

Before encoding, a readable copy of the dataframe (`df_viz`) was preserved with the original human-readable category labels, specifically for use in visualizations and hypothesis testing where readability matters more than numeric encoding.

Keep a Readable Copy Before Encoding

`city` and `profession` will be one-hot encoded and `gender` label-encoded next, which makes them hard to read in plots/tables. We keep a copy (`df_viz`) now, with the original readable labels, to use for visualization and hypothesis testing.

```
df_viz = df.copy()
```

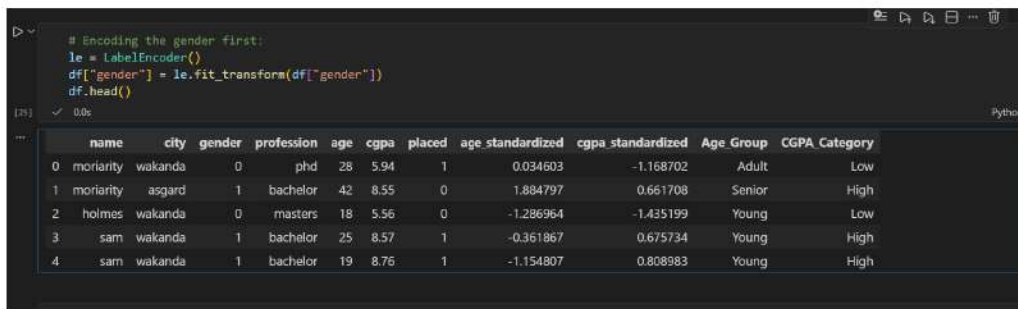
Handling with Categorical Data: Encoding Techniques

```
df.head()
```

	name	city	gender	profession	age	cgpa	placed	age_standardized	cgpa_standardized	Age_Group	CGPA_Category
0	marianity	wakanda	female	phd	28	5.94	1	0.034603	-1.168702	Adult	Low
1	marianity	asgard	male	bachelor	42	8.55	0	1.884797	0.661708	Senior	High
2	holmes	wakanda	female	masters	18	5.56	0	-1.280964	-1.435199	Young	Low
3	sam	wakanda	male	bachelor	25	8.57	1	-0.361867	0.675734	Young	High
4	sam	wakanda	male	bachelor	19	8.76	1	-1.154807	0.808983	Young	High

Figure 12: Preserving a readable copy (`df_viz`) before encoding

`gender`, a binary categorical variable, was transformed using **Label Encoding** (scikit-learn's `LabelEncoder`), mapping female $\rightarrow$ 0 and male $\rightarrow$ 1.

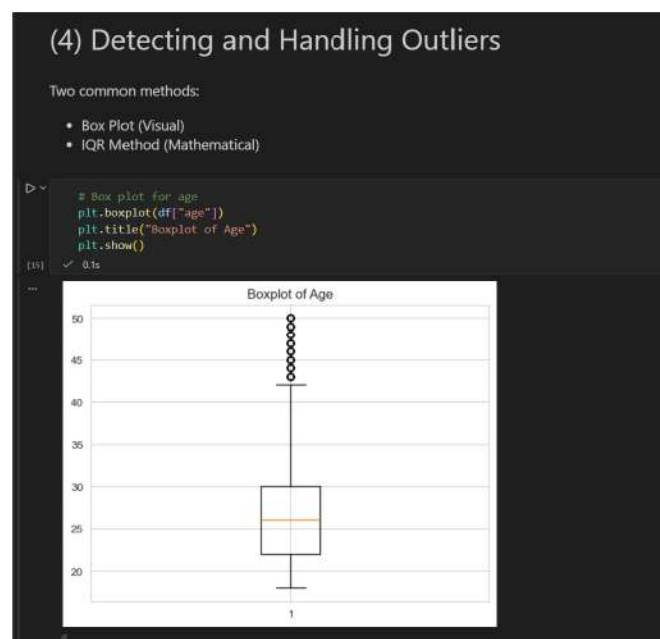
Figure 13: Label encoding applied to **gender**

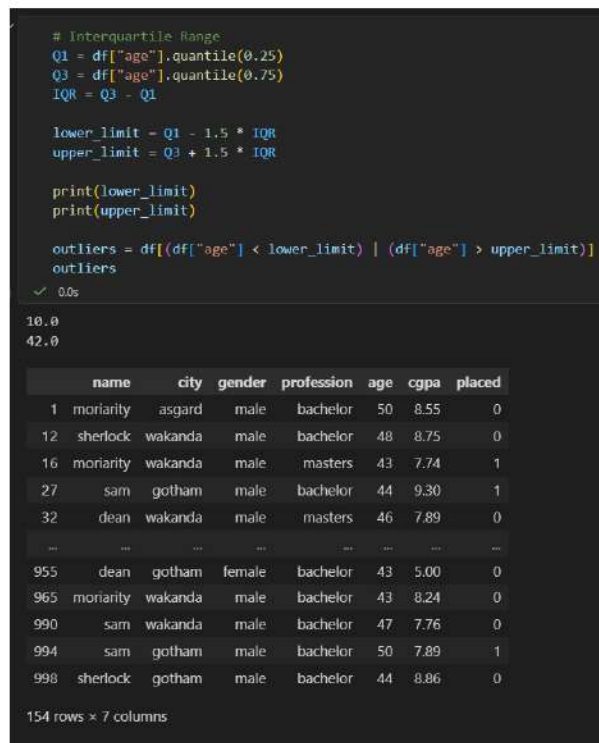
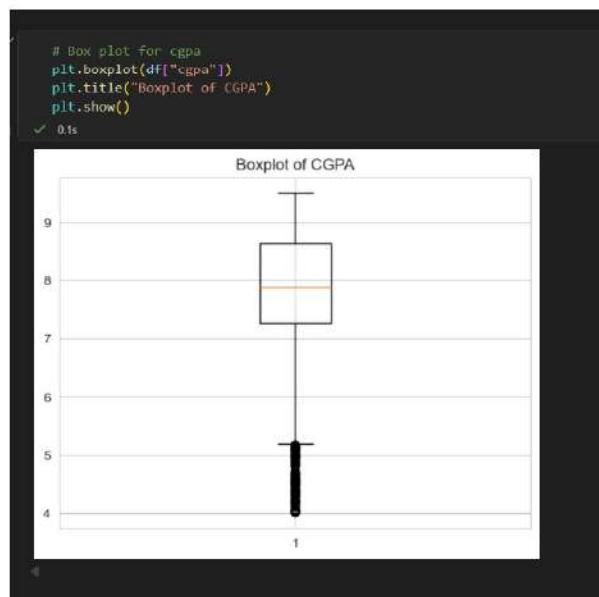
city and **profession**, both nominal categorical variables with more than two categories, were transformed using **One-Hot Encoding** (`pd.get_dummies`), avoiding the false ordinal relationship that label encoding would have implied for these columns.

Figure 14: One-hot encoding of **city** and **profession**

4.5 e. Outlier Detection

Outliers in the two continuous variables (**age**, **cgpa**) were detected using two complementary methods: visual inspection via box plots, and the mathematical Interquartile Range (IQR) rule (values beyond $Q1 - 1.5 \times IQR$ or $Q3 + 1.5 \times IQR$).

Figure 15: Box plot of **age**, showing high-end outliers above ~ 42 years

Figure 16: IQR bounds for `age` (10.0, 42.0) and the 154 rows identified as outliersFigure 17: Box plot of CGPA, showing a cluster of low-end outliers below ~ 5.2

Rather than deleting outlier rows outright (which would have discarded a further $\sim 15\%$ of the data), extreme values were clipped to the IQR boundary limits using `df["age"] = df["age"].clip(lower_limit, upper_limit)`. A follow-up check confirmed zero rows remained outside the bounds after clipping.

```
# Instead of removing rows, we replace extreme values with the boundary values.
df["age"] = df["age"].clip(lower_limit, upper_limit)
✓ 0.0s
```

```
out_of_bounds = df[(df["age"] < lower_limit) | (df["age"] > upper_limit)]
print(f"Number of rows outside limits: {len(out_of_bounds)}")
✓ 0.0s
```

Number of rows outside limits: 0

```
df.describe()
```

	age	cgpa	placed
count	1035.000000	1035.000000	1035.000000
mean	27.738164	7.606464	0.496618
std	7.570433	1.426599	0.500230
min	18.000000	4.010000	0.000000
25%	22.000000	7.260000	0.000000
50%	26.000000	7.890000	0.000000
75%	30.000000	8.640000	1.000000
max	42.000000	9.490000	1.000000

Figure 18: Clipping outliers to IQR boundaries and confirming zero out-of-bound rows

After all cleaning and outlier handling, the final numeric summary (age: mean 27.74, range 18–42; cgpa: mean 7.61, range 4.01–9.49; placed: mean 0.497) reflects a clean, well-behaved dataset of 1,035 rows ready for analysis.

5 Data Visualization

A range of univariate and bivariate plots were generated using `matplotlib` and `seaborn` to explore the distribution of each feature and its relationship with the placement outcome.

5.1 Univariate Distributions

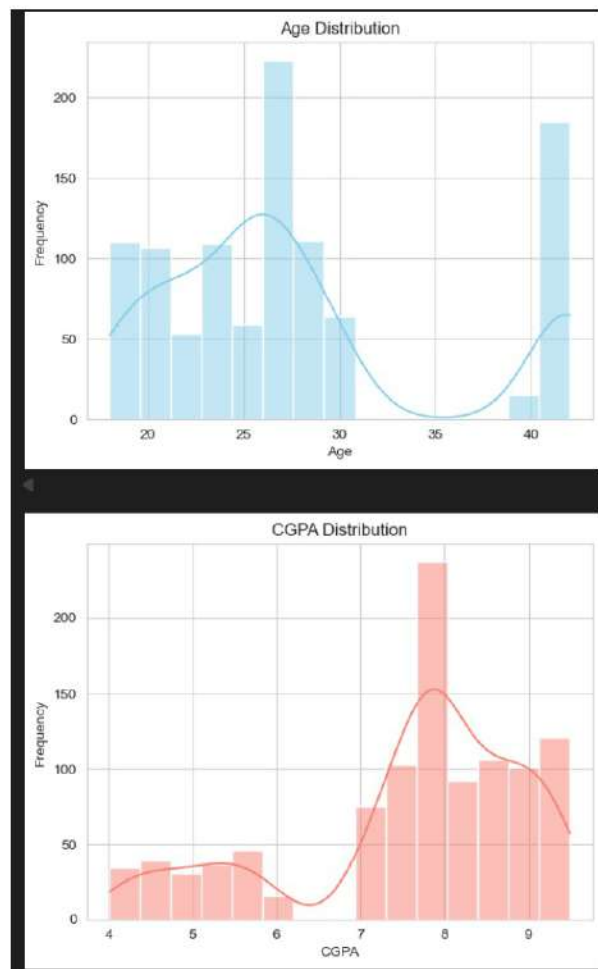


Figure 19: Distribution of Age (mildly bimodal, with a secondary cluster around 40–42) and CGPA (right-skewed, with a secondary low-scoring cluster below 6)

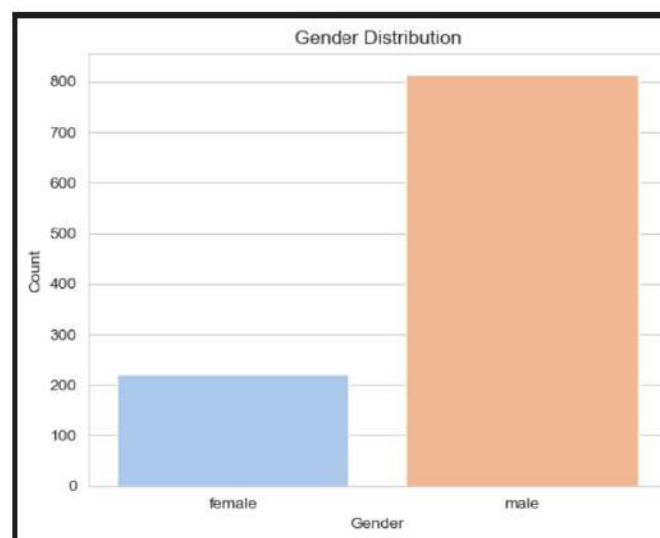


Figure 20: Gender distribution: males (~ 813) considerably outnumber females (~ 220) in the dataset

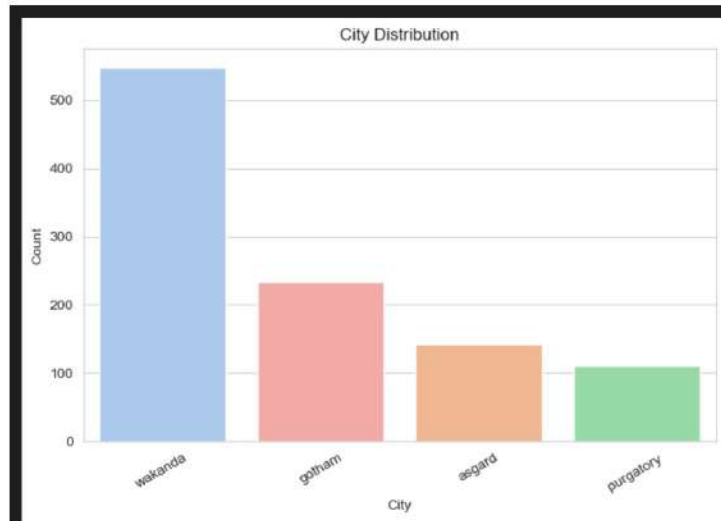


Figure 21: City distribution: Wakanda is the dominant city (~547 students), followed by Gotham, Asgard, and Purgatory

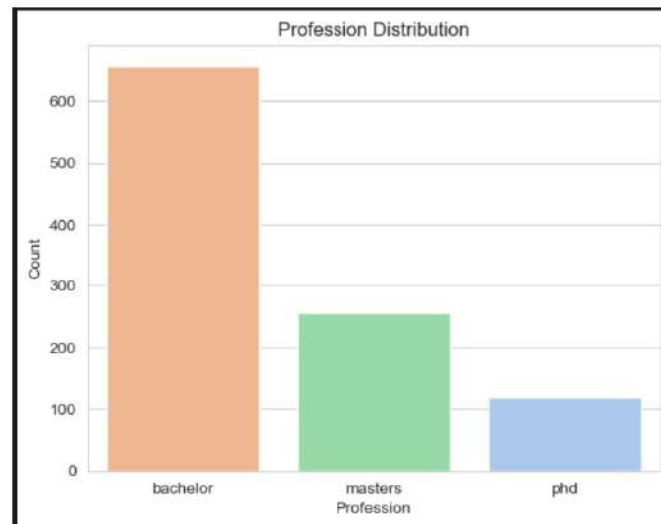


Figure 22: Profession (degree level) distribution: Bachelor's students dominate (~657), followed by Master's (~254) and PhD (~119)

The target variable itself is balanced: 521 students not placed vs. 514 placed (50.3% vs. 49.7%), which avoids the class-imbalance issues that can complicate group comparisons.

5.2 Placement vs. Categorical Features

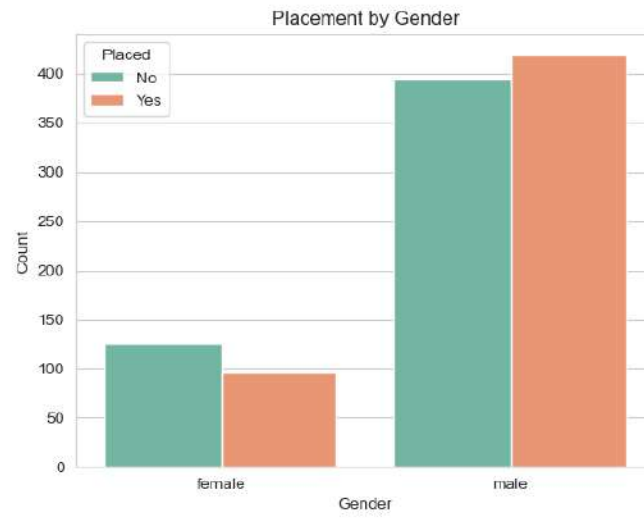


Figure 23: Placement counts by Gender

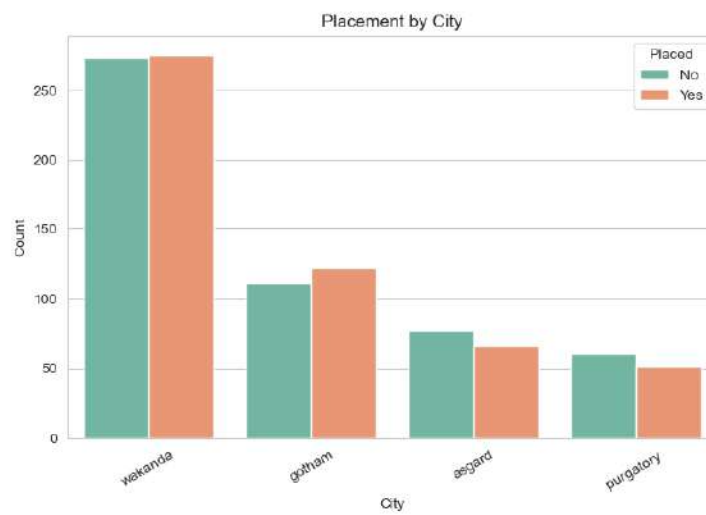


Figure 24: Placement counts by City

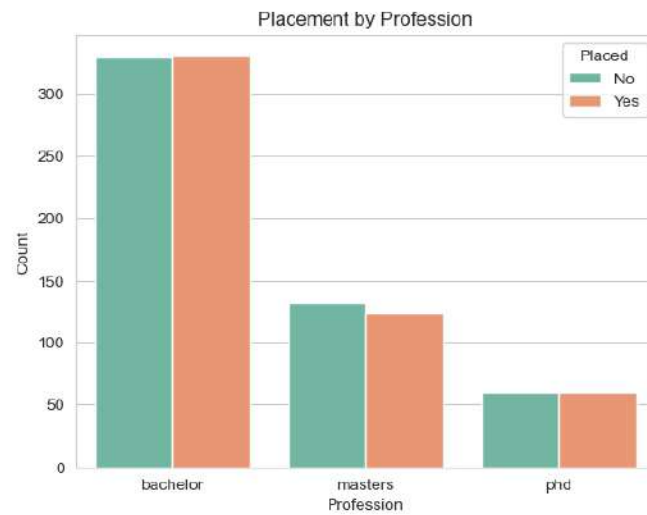


Figure 25: Placement counts by Profession

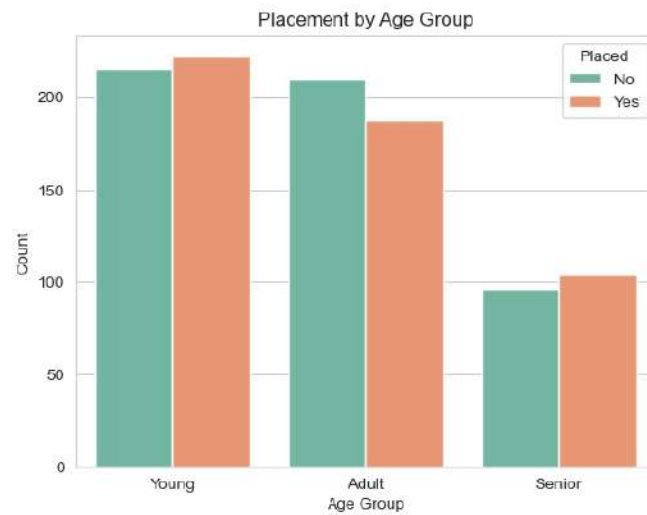


Figure 26: Placement counts by Age Group

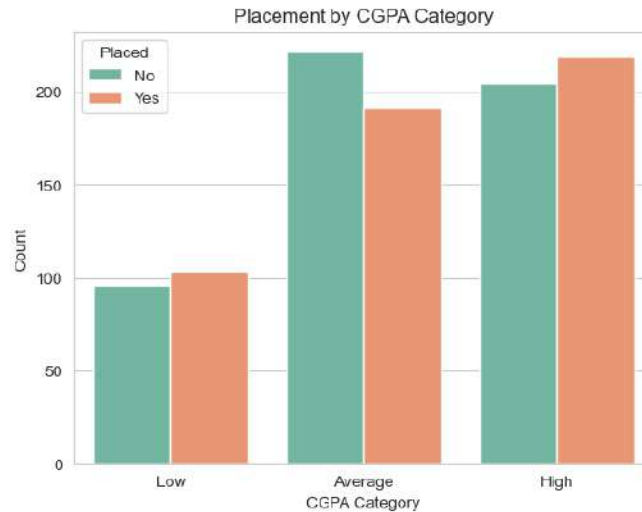


Figure 27: Placement counts by CGPA Category

The placement rate (% placed) within each category, computed with `groupby().mean()`, was:

Table 2: Placement rate by Gender

Gender	Placement Rate
Female	42.99%
Male	51.47%

Table 3: Placement rate by Profession

Profession	Placement Rate
Bachelor	50.08%
Masters	48.44%
PhD	50.00%

Table 4: Placement rate by City

City	Placement Rate
Asgard	46.15%
Gotham	52.36%
Purgatory	45.95%
Wakanda	50.18%

Gender shows the widest gap in raw placement rate (male 51.5% vs. female 43.0%) among the categorical breakdowns, foreshadowing the statistically significant chi-square result found in Section 7.

5.3 Placement vs. Numeric Features



Figure 28: Age vs. CGPA scatter plot, colored by placement status — no visible separation between the two placement groups

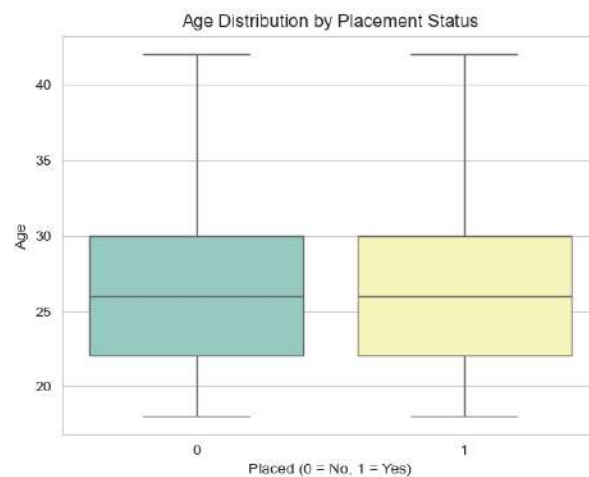


Figure 29: Age distribution by placement status (box plot) — near-identical medians and spread

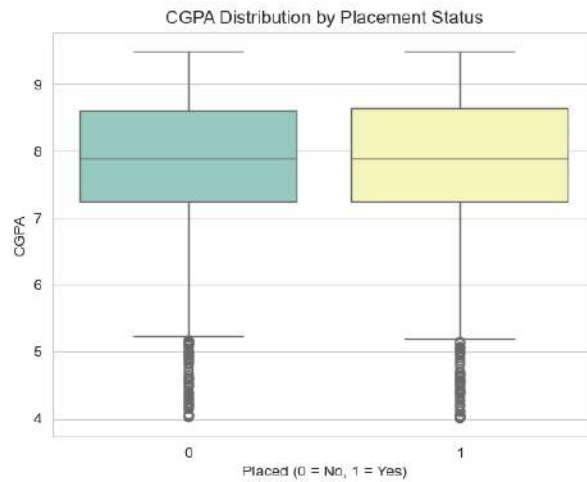


Figure 30: CGPA distribution by placement status (box plot) — near-identical medians and spread

The descriptive statistics confirm the near-total overlap visible in the box plots:

Age summary by Placement:								
	count	mean	std	min	25%	50%	75%	max
placed								
0	521.0	27.669866	7.404419	18.0	22.0	26.0	30.0	42.0
1	514.0	27.807393	7.741679	18.0	22.0	26.0	30.0	42.0

CGPA summary by Placement:								
	count	mean	std	min	25%	50%	75%	max
placed								
0	521.0	7.601190	1.406067	4.03	7.26	7.89	8.61	9.49
1	514.0	7.611809	1.448465	4.01	7.26	7.89	8.64	9.49

Figure 31: Age and CGPA summary statistics grouped by placement status

Statistical Analysis

Hypothesis Testing and Correlation

a. Hypothesis Testing

✓ **Test 1: Independent t-test - CGPA vs Placed**

Question: Is there a statistically significant difference in CGPA between students who got placed and those who didn't?

- **H0 (Null Hypothesis):** There is no significant difference in mean CGPA between placed and not-placed students.
- **H1 (Alternative Hypothesis):** There is a significant difference in mean CGPA between placed and not-placed students.
- **Significance level (alpha):** 0.05

```

cgpa_placed = df[df["placed"] == 1]["cgpa"]
cgpa_not_placed = df[df["placed"] == 0]["cgpa"]

t_stat, p_value = stats.ttest_ind(cgpa_placed, cgpa_not_placed)

print(f"T-statistic: {t_stat:.4f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("\nResult: Reject H0 -> Significant difference in CGPA between placed and not-placed students.")
else:
    print("\nResult: Fail to reject H0 -> No significant difference in CGPA between placed and not-placed students.")
  
```

35] ✓ 0.05

T-statistic: 0.1197
P-value: 0.9048

Result: Fail to reject H0 -> No significant difference in CGPA between placed and not-placed students.

Figure 33: Independent t -test, CGPA vs. Placed: $t = 0.1197$, $p = 0.9048$

Result: $p = 0.9048 > 0.05 \Rightarrow$ **Fail to reject** H_0 . No statistically significant difference in CGPA between placed and not-placed students.

Test 2: Independent t -test — Age vs. Placed

H_0 : There is no significant difference in mean Age between placed and not-placed students.

H_1 : There is a significant difference in mean Age between placed and not-placed students.

✓ **Test 2: Independent t-test - Age vs Placed**

Question: Is there a statistically significant difference in Age between students who got placed and those who didn't?

- **H0 (Null Hypothesis):** There is no significant difference in mean Age between placed and not-placed students.
- **H1 (Alternative Hypothesis):** There is a significant difference in mean Age between placed and not-placed students.
- **Significance level (alpha):** 0.05

```

age_placed = df[df["placed"] == 1]["age"]
age_not_placed = df[df["placed"] == 0]["age"]

t_stat, p_value = stats.ttest_ind(age_placed, age_not_placed)

print(f"T-statistic: {t_stat:.4f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("\nResult: Reject H0 -> Significant difference in Age between placed and not-placed students.")
else:
    print("\nResult: Fail to reject H0 -> No significant difference in Age between placed and not-placed students.")
  
```

36] ✓ 0.05

T-statistic: 0.2921
P-value: 0.7703

Result: Fail to reject H0 -> No significant difference in Age between placed and not-placed students.

Figure 34: Independent t -test, Age vs. Placed: $t = 0.2921$, $p = 0.7703$

Result: $p = 0.7703 > 0.05 \Rightarrow$ **Fail to reject** H_0 . No statistically significant difference in Age between placed and not-placed students.

5.4 Correlation Heatmap

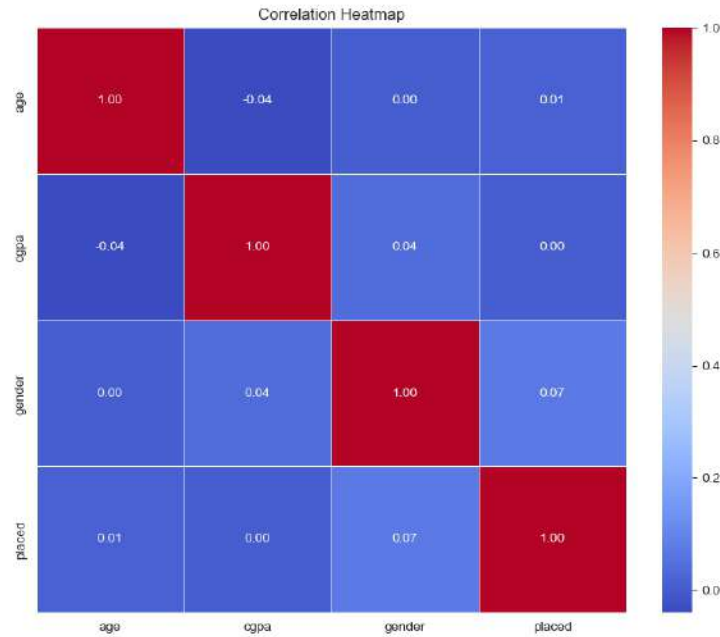


Figure 32: Correlation heatmap of `age`, `cgpa`, `gender`, and `placed`

All pairwise correlations among `age`, `cgpa`, `gender`, and `placed` are very close to zero (largest magnitude: 0.07 between `gender` and `placed`), visually confirming the absence of any strong linear relationship among these variables.

6 Statistical Analysis

Beyond visual inspection, formal statistical hypothesis tests were used to determine whether observed differences and associations in the data are statistically significant ($\alpha = 0.05$) or could plausibly be due to random chance.

6.1 a. Hypothesis Testing

Test 1: Independent *t*-test — CGPA vs. Placed

H_0 : There is no significant difference in mean CGPA between placed and not-placed students.

H_1 : There is a significant difference in mean CGPA between placed and not-placed students.

Statistical Analysis

Hypothesis Testing and Correlation

a. Hypothesis Testing

✓ **Test 1: Independent t-test - CGPA vs Placed**

Question: Is there a statistically significant difference in CGPA between students who got placed and those who didn't?

- **H0 (Null Hypothesis):** There is no significant difference in mean CGPA between placed and not-placed students.
- **H1 (Alternative Hypothesis):** There is a significant difference in mean CGPA between placed and not-placed students.
- **Significance level (alpha):** 0.05

```

cgpa_placed = df[df["placed"] == 1]["cgpa"]
cgpa_not_placed = df[df["placed"] == 0]["cgpa"]

t_stat, p_value = stats.ttest_ind(cgpa_placed, cgpa_not_placed)

print(f"T-statistic: {t_stat:.4f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("\nResult: Reject H0 -> Significant difference in CGPA between placed and not-placed students.")
else:
    print("\nResult: Fail to reject H0 -> No significant difference in CGPA between placed and not-placed students.")
  
```

35] ✓ 0.05

T-statistic: 0.1197
P-value: 0.9048

Result: Fail to reject H0 -> No significant difference in CGPA between placed and not-placed students.

Figure 33: Independent t -test, CGPA vs. Placed: $t = 0.1197$, $p = 0.9048$

Result: $p = 0.9048 > 0.05 \Rightarrow$ **Fail to reject** H_0 . No statistically significant difference in CGPA between placed and not-placed students.

Test 2: Independent t -test — Age vs. Placed

H_0 : There is no significant difference in mean Age between placed and not-placed students.

H_1 : There is a significant difference in mean Age between placed and not-placed students.

✓ **Test 2: Independent t-test - Age vs Placed**

Question: Is there a statistically significant difference in Age between students who got placed and those who didn't?

- **H0 (Null Hypothesis):** There is no significant difference in mean Age between placed and not-placed students.
- **H1 (Alternative Hypothesis):** There is a significant difference in mean Age between placed and not-placed students.
- **Significance level (alpha):** 0.05

```

age_placed = df[df["placed"] == 1]["age"]
age_not_placed = df[df["placed"] == 0]["age"]

t_stat, p_value = stats.ttest_ind(age_placed, age_not_placed)

print(f"T-statistic: {t_stat:.4f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("\nResult: Reject H0 -> Significant difference in Age between placed and not-placed students.")
else:
    print("\nResult: Fail to reject H0 -> No significant difference in Age between placed and not-placed students.")
  
```

36] ✓ 0.05

T-statistic: 0.2921
P-value: 0.7703

Result: Fail to reject H0 -> No significant difference in Age between placed and not-placed students.

Figure 34: Independent t -test, Age vs. Placed: $t = 0.2921$, $p = 0.7703$

Result: $p = 0.7703 > 0.05 \Rightarrow$ **Fail to reject** H_0 . No statistically significant difference in Age between placed and not-placed students.

Test 3: Chi-square Test — Gender vs. Placed

H_0 : Gender and Placement are independent (no association).

H_1 : Gender and Placement are dependent (there is an association).

```

~ Test 3: Chi-square Test - Gender vs Placed

Question: Is there a statistically significant association between Gender and Placement status?

• H0 (Null Hypothesis): Gender and Placement are independent (no association).
• H1 (Alternative Hypothesis): Gender and Placement are dependent (there is an association).
• Significance level (alpha): 0.05

contingency_table_gender = pd.crosstab(df_viz["gender"], df_viz["placed"])
print("Contingency Table:")
print(contingency_table_gender)

chi2_stat, p_value, dof, expected = stats.chi2_contingency(contingency_table_gender)

print(f"\nChi-square statistic: {chi2_stat:.4f}")
print(f"Degrees of freedom: {dof}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("\nResult: Reject H0 -> Significant association between Gender and Placement.")
else:
    print("\nResult: Fail to reject H0 -> No significant association between Gender and Placement.")

```

[37] ✓ 0.0s

```

--- Contingency Table:
placed  0    1
gender
female 126   95
male   395  419

Chi-square statistic: 4.6751
Degrees of freedom: 1
P-value: 0.0306

Result: Reject H0 -> Significant association between Gender and Placement.

```

Figure 35: Chi-square test, Gender vs. Placed: $\chi^2 = 4.6751$, $df = 1$, $p = 0.0306$

Result: $p = 0.0306 < 0.05 \Rightarrow$ **Reject** H_0 . There is a statistically significant association between Gender and Placement.

Test 4: Chi-square Test — Profession vs. Placed

H_0 : Profession and Placement are independent (no association).

H_1 : Profession and Placement are dependent (there is an association).

Test 4: Chi-square Test - Profession vs Placed

Question: Is there a statistically significant association between Profession (degree level) and Placement status?

- **H0 (Null Hypothesis):** Profession and Placement are independent (no association).
- **H1 (Alternative Hypothesis):** Profession and Placement are dependent (there is an association).
- **Significance level (alpha):** 0.05

```

contingency_table_prof = pd.crosstab(df_viz["profession"], df_viz["placed"])
print("Contingency Table:")
print(contingency_table_prof)

chi2_stat, p_value, dof, expected = stats.chi2_contingency(contingency_table_prof)

print(f"\nChi-square statistic: {chi2_stat:.4f}")
print(f"Degrees of freedom: {dof}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("\nResult: Reject H0 -> Significant association between Profession and Placement.")
else:
    print("\nResult: Fail to reject H0 -> No significant association between Profession and Placement.")

```

[30] ✓ 0.0s

```

... Contingency Table:
placed      0      1
profession
bachelor    329    330
masters     132    124
phd          60     60

Chi-square statistic: 0.2042
Degrees of freedom: 2
P-value: 0.9029

Result: Fail to reject H0 -> No significant association between Profession and Placement.

```

Figure 36: Chi-square test, Profession vs. Placed: $\chi^2 = 0.2042$, $df = 2$, $p = 0.9029$

Result: $p = 0.9029 > 0.05 \Rightarrow$ **Fail to reject** H_0 . No statistically significant association between Profession and Placement.

Summary of Hypothesis Tests

Table 5: Summary of hypothesis test results

Test	Statistic	p -value	Conclusion ($\alpha = 0.05$)
CGPA vs Placed (t -test)	$t = 0.1197$	0.9048	Not significant
Age vs Placed (t -test)	$t = 0.2921$	0.7703	Not significant
Gender vs Placed (χ^2)	$\chi^2 = 4.6751$	0.0306	Significant
Profession vs Placed (χ^2)	$\chi^2 = 0.2042$	0.9029	Not significant

6.2 b. Correlation Analysis

Pearson correlation coefficients were computed for each pair of numeric variables, along with their statistical significance:

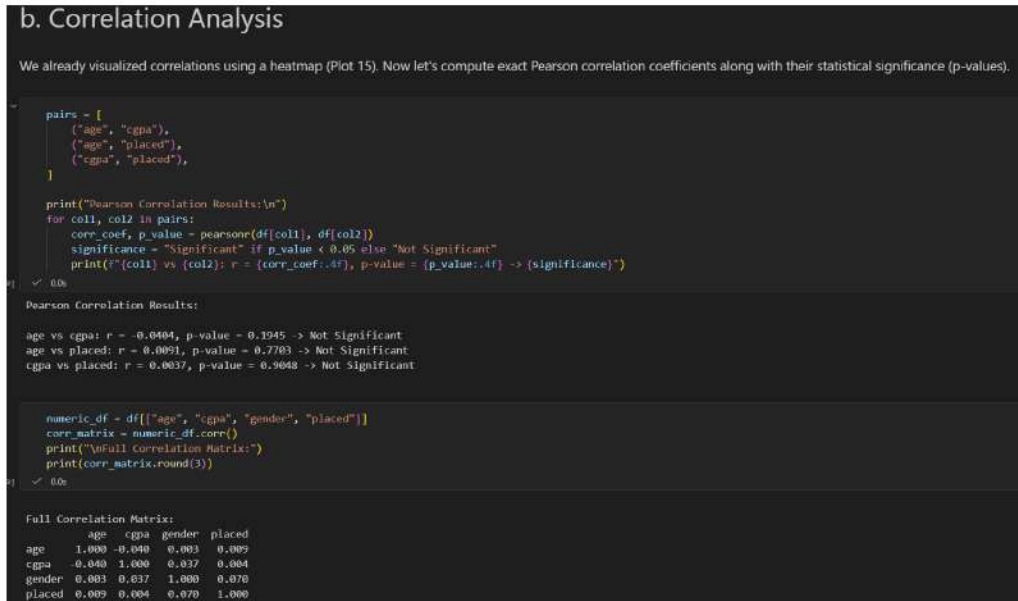


Figure 37: Pearson correlation coefficients and full correlation matrix

Table 6: Pearson correlation summary

Pair	Pearson r	p -value	Significant?
Age vs CGPA	-0.0404	0.1945	No
Age vs Placed	0.0091	0.7703	No
CGPA vs Placed	0.0037	0.9048	No
Gender vs Placed	0.070	—	Consistent w/ χ^2 result

Every correlation coefficient is close to zero ($|r| < 0.08$), confirming that none of the numeric variables share a meaningful linear relationship with each other or with the placement outcome, consistent with the heatmap in Figure 11 and the hypothesis test results above.

7 Results and Discussion

Combining the visual exploration and formal statistical testing, the following conclusions emerge from this dataset:

- **Gender** is the only feature with a statistically significant association with placement ($\chi^2 = 4.6751$, $p = 0.0306$). Male students show a placement rate of 51.5% versus 43.0% for female students — a gap of roughly 8.5 percentage points.
- **CGPA** shows no significant relationship with placement, either via t -test ($p = 0.9048$) or correlation ($r = 0.0037$, $p = 0.9048$). Students who were placed and not placed have virtually identical CGPA distributions (means of 7.61 vs. 7.60).
- **Age** shows no significant relationship with placement, either via t -test ($p = 0.7703$) or correlation ($r = 0.0091$, $p = 0.7703$).
- **Profession** (degree level) shows no significant association with placement ($\chi^2 = 0.2042$, $p = 0.9029$); bachelor's, master's, and PhD students are placed at almost identical rates (~ 48 –50%).

- **City** shows a modest spread in raw placement rates (45.9% to 52.4%) but was not formally tested; given the pattern seen with profession, this spread likely reflects sampling noise rather than a true effect.
- **Age and CGPA** are themselves uncorrelated ($r = -0.0404$, not significant), indicating these two numeric features vary independently of one another in this dataset.

Overall, the data suggests that in this dataset, placement outcome behaves close to a random 50/50 split with respect to academic performance (CGPA), age, and degree level — the only detectable (and still modest) signal is a gender-based difference. This is a somewhat counter-intuitive finding relative to common assumptions (e.g., that CGPA strongly predicts placement), and is worth flagging explicitly rather than assuming CGPA “must” matter.

Given the earlier observation that the dataset’s names and cities are drawn from fictional universes, it is plausible that placement outcomes here were generated with a random or near-random process (possibly with a small deliberate gender effect), rather than reflecting a genuine underlying real-world mechanism. This context should be kept in mind: the statistical conclusions are valid descriptions of this dataset, but should not be over-generalized to real placement processes without validation on real institutional data.

8 Challenges and Future Enhancement

Challenges Encountered

- High missing-value rates (8%–18%) across nearly every column required careful, column-specific imputation strategy rather than simple row deletion, to avoid losing a large share of the dataset.
- A meaningful number of exact duplicate rows (5.9%) had to be identified and removed to avoid double-counting records in downstream statistics.
- Right- and left-skewed distributions (age’s bimodal shape; CGPA’s low-end cluster) required outlier handling (IQR clipping) rather than naive z -score filtering, to avoid discarding legitimate data.
- The weak-to-negligible correlations and mostly non-significant test results meant the dataset offered limited explanatory power for placement — a reminder that clean methodology can still yield a “no strong effect” conclusion, which is itself a valid and useful finding.
- Categorical encoding choices (label vs. one-hot) had to be made carefully to avoid introducing false ordinal relationships into nominal variables like `city`.

Future Enhancements

- Incorporate richer features that plausibly drive placement in the real world — interview performance, technical/soft-skill assessments, internship experience, extracurricular involvement, and communication skills.
- Apply predictive machine learning models (e.g., Logistic Regression, Random Forest, Gradient Boosting) to formally test predictive power and quantify feature importance, going beyond pairwise hypothesis tests to a multivariate model.
- Use multivariate/interaction analysis (e.g., two-way ANOVA, interaction terms) to check whether combinations of features (e.g., gender $\times$ profession) reveal effects not visible in single-variable tests.

- Validate findings on a real, verified institutional placement dataset to confirm whether the gender effect observed here generalizes, and to check it does not stem from confounding or sampling artifacts.
- Apply more advanced missing-data techniques (e.g., KNN or iterative/MICE imputation) instead of simple median/mode imputation, to better preserve relationships between correlated features when imputing.
- Build an interactive dashboard (e.g., in Power BI, Tableau, or a Streamlit app) to allow stakeholders to explore placement trends dynamically across cities, professions, and demographic segments.